Validation using Data Annotations in ASP.NET Core Web API

Back to: ASP.NET Core Web API Tutorials

Validation using Data Annotations in ASP.NET Core Web API

In this article, I will discuss how to implement server-side **validation using Data Annotations in ASP.NET Core Web API** Applications with Examples. Please read our previous article discussing **How to Exclude Properties from Model Binding in ASP.NET Core Web API** Applications with Examples.

What is Validation?

Validation is the process of verifying that the data entering your application is correct, meaningful, and safe to use. In simple terms, it's like a **Security Checkpoint** for your data; only properly formatted, acceptable data is allowed to move forward to the business logic or the database. When a client (like a browser, mobile app, or another system) sends data to an ASP.NET Core Web API application, validation ensures that:

- Required fields are not missing.
- Data is in the expected format (e.g., email looks like abc@example.com).
- Values fall within allowed ranges (e.g., Age must be between 18 and 60).
- Strings do not exceed length limits (e.g., username up to 50 characters).

Without validation, your application might process **Wrong**, **Incomplete**, or even **Malicious** data, leading to **Runtime Errors, Data Corruption, or Security Breaches**.

Example: If your API accepts a UserRegistrationRequest with properties like Name, Age, Email, you want to ensure:

- Name is Not Empty
- Age is within a specified Range
- Email is Validly Formatted

In **ASP.NET Core Web API**, validation typically happens when data is sent to your API via JSON and is bound to a model (DTO). The framework can then automatically validate this model against rules defined using Data Annotations or custom logic.

Why Do We Need Validation in ASP.NET Core Web API?

ASP.NET Core Web API acts as the **Central Point of Communication** between clients and your backend. Any malicious or incorrect data that reaches the server can cause serious problems. Here's why validation is so crucial:

Data Integrity

Validation ensures that only **Valid and Meaningful Data** is stored in the database. For example:

- Prevents saving an empty "Name" field for a student.
- Ensures "Price" is not negative in a product record.
- "Date of Joining" cannot be before "Date of Birth".

By enforcing these rules, your database remains clean, consistent, and trustworthy. Without validation, invalid or incomplete data can lead to incorrect reports, calculation errors, and broken relationships between tables.

Improved User Experience

Validation allows the system to **Give Users Immediate, Helpful Feedback** when they provide incorrect inputs.

- For example, if a user submits a form with missing or invalid fields, meaningful validation messages like *"***Email address is required***"* or *"***Age must be between 18 and 60***"* guide them to fix issues without confusion.
- This clarity helps users fix their mistakes quickly and reduces frustration, improving overall usability and trust in the system.

Security

Proper validation of input fields also prevents common security threats. For instance:

- **SQL Injection:** Ensuring only valid and expected inputs reach your SQL queries by validating inputs and using parameterized queries.
- **Cross-Site Scripting (XSS):** It helps avoid Cross-Site Scripting (XSS) by sanitizing dangerous scripts in text inputs.
- **Cross-Site Request Forgery (CSRF):** It prevents **Cross-Site Request Forgery (CSRF)** when validation is combined with anti-forgery tokens.

By validating and sanitizing incoming data, we make it much harder for attackers to exploit vulnerabilities in your system. So, validation doesn't just protect your data, it protects your entire system.

Error Prevention

Without validation, Invalid or unexpected input can cause **Runtime Exceptions**, such as:

- Null reference errors
- Overflow or conversion errors
- Database constraint violations

By catching problems early in the request pipeline, we:

- Avoid unnecessary processing.
- Reduce debugging time.
- Keep our logs cleaner and more meaningful.

So, validation also acts as a **filter,** keeping our business logic clean and our application stable, preventing costly and confusing runtime errors.

Types of Validation

Validation can occur at different stages in an application's data flow, client, server, and database. Each layer adds an extra level of protection. Let's break them down:

Client-Side Validation

This happens on the **client.** It provides **Instant Feedback**. Users see validation messages before the data even leaves their device.

- Happens in the **Browser or Mobile App** before data is sent to the server.
- Uses JavaScript, HTML5 attributes, or frameworks like Angular/React.
- Provides **Instant Feedback** (e.g., red borders, tooltips).

Example: When a user types an invalid email address, the form instantly highlights it and displays **"Please enter a valid email address.**"

Pros:

- Fast and User-Friendly.
- Reduces Unnecessary Requests to the Server.

Cons:

- Can be bypassed easily using tools like Postman or browser dev tools.
- You can also bypass these validations by disabling JavaScript in the browser.
- Hence, it **cannot Be Trusted Alone** for security or data integrity.

Server-Side Validation

This happens on the **Server side**. The server checks every piece of information before saving or processing it.

- Happens in the **API** after receiving the request from the client.
- Ensures **Business Rules** and **Data Integrity** before any processing.
- Cannot be bypassed by the user.
- Implemented using **Data Annotations**, **Fluent Validation**, or **Custom Logic**.

Example: Even if a hacker manually sends an invalid email via Postman, your Web API validates it before inserting it into the database.

Pros:

- Reliable and Secure.
- Enforces Business Rules and Data Consistency.
- Cannot be bypassed by clients.

This is the **Most Important** form of validation in an ASP.NET Core Web API.

Database Validation

This is the Final Line of Defense for data insertion or updates, enforced directly at the Database Schema Level. It includes constraints like:

- **NOT NULL** → ensures required columns aren't left blank.
- **CHECK (Age >= 18)** → ensures logical value constraints.
- **UNIQUE** → prevents duplicate records (like duplicate email addresses).
- **Foreign Key** → Managing the Relationship by checking the data existence.

Even if both client-side and server-side validations fail to catch an issue, these constraints stop invalid data from being saved into the database.

Note: Always remember, **Client-side validation is for convenience, but server-side validation is for safety.** And database validation is the **final safety net**.

Different Ways to Implement Server-Side Validations in ASP.NET Core Web API

ASP.NET Core provides multiple ways to validate incoming data at the server level. Each method has its own use case and flexibility.

Model Validation Using Data Annotations

This is the simplest and most common method. You apply attributes directly to your model properties, such as:

```
public class StudentDTO
{
    [Required(ErrorMessage = "Name is required")]
    [StringLength(50, ErrorMessage = "Name can't exceed 50 characters")]
    public string Name { get; set; }

    [Range(18, 60, ErrorMessage = "Age must be between 18 and 60")]
    public int Age { get; set; }

    [EmailAddress(ErrorMessage = "Invalid Email Address")]
    public string Email { get; set; }
}
```

When ASP.NET Core binds the request body to this model, it automatically checks these attributes and populates the ModelState. If invalid, you can return a 400 Bad Request with validation errors, without writing any manual logic.

Best for: Simple field-level checks.

Fluent Validation

FluentValidation is a third-party library that provides a **Fluent (Code-Based)** way to define complex validation rules. It's great for **Advanced Scenarios, Cross-Field Checks, or Reusable Validation Logic**. For example:

```
public class StudentValidator : AbstractValidator<StudentDTO>
{
    public StudentValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty().WithMessage("Name is required")
            .Length(3, 50).WithMessage("Name must be between 3 and 50
characters");

        RuleFor(x => x.Email)
            .EmailAddress().WithMessage("Please enter a valid email");

        RuleFor(x => x.Age)
            .InclusiveBetween(18, 60)
            .WithMessage("Age must be between 18 and 60");
    }
}
```

Best For: Enterprise applications or complex validation logic that needs reusability.

Manual Validation

Sometimes, you may need custom or conditional logic that can't be expressed with annotations or FluentValidation. For example:

- Checking if a username already exists in the database.
- Ensuring a combination of fields is unique.
- Validating based on business rules (e.g., if IsPremiumUser == true, then PaymentDetails must be provided).

Example:

```
if (string.IsNullOrEmpty(student.Name))
    return BadRequest("Name cannot be empty");

if (_studentService.IsEmailAlreadyUsed(student.Email))
    return BadRequest("Email already exists");
```

Best for: Dynamic, business-specific validations or validations involving database lookups.

What Are Data Annotation Attributes in ASP.NET Core Web API?

In ASP.NET Core Web API, **Data Annotation Attributes** are **Decorators** (special metadata applied to class properties) that enforce rules directly at the model level. When the client sends data, the ASP.NET Core **Model Binding and Validation System** automatically checks these attributes and populates the ModelState object.

If validation fails, the controller can immediately return an error response; no manual checks are needed. So, essentially, these attributes act as **Automated Gatekeepers,** ensuring that data from the client side is valid and meaningful before any business logic executes.

Example:

```
public class StudentDTO
{
    [Required(ErrorMessage = "Name is required.")]
    [StringLength(50, ErrorMessage = "Name must be less than 50 characters.")]
    public string Name { get; set; }

    [Range(18, 60, ErrorMessage = "Age must be between 18 and 60.")]
    public int Age { get; set; }
}
```

If a request body misses the Name or provides an Age less than 18, the framework automatically rejects it and returns:

```
{
  "errors": {
    "Name": ["Name is required."],
    "Age": ["Age must be between 18 and 60."]
  }
}
```

This automatic behaviour makes **Data Annotations** one of the simplest and most efficient validation techniques in ASP.NET Core.

Commonly Used Data Annotation Attributes

Let's now explore each attribute deeply, when and why we would use it. The validation attributes belong to System.ComponentModel.DataAnnotations namespace.

[Required]

The [Required] attribute ensures that a property must have a value; it cannot be null, empty, or omitted from the request payload. It's typically applied to essential fields such as FirstName, Email, Password, or JoiningDate, where missing values would render the record incomplete.

Example:
[Required(ErrorMessage = "First Name is required.")]
public string FirstName { get; set; }

How it works: During model binding, ASP.NET Core checks if the FirstName property is present and non-empty. If not, it automatically marks the ModelState as invalid and returns a 400 Bad Request response when using [ApiController].

[StringLength]

The [StringLength] attribute defines both the **Minimum and Maximum Number of Characters** allowed in a string property. This helps prevent overly long or too-short text input, ensuring consistency and avoiding storage or UI layout issues. It is used for text fields such as FirstName, LastName, or Designation to maintain clean, standardized user data.

Example:
[StringLength(50, MinimumLength = 3, ErrorMessage = "First Name must be between 3 and 50

characters.")]
public string FirstName { get; set; }

How it works: ASP.NET Core validates that the string's length falls within the specified range.

[MaxLength] and [MinLength]

These attributes validate either the maximum or minimum number of characters allowed, particularly useful when only one limit matters.

Example:
[MaxLength(1000, ErrorMessage = "Address cannot exceed 100 characters.")]
public string Address { get; set; }

[MinLength(6, ErrorMessage = "Password must be at least 6 characters long.")]
public string Password { get; set; }

When to use:

- Use [MaxLength] for properties like Address or City where you only want to restrict excessive input.
- Use [MinLength] for fields like Password or Username to ensure adequate security or readability.

[Range]

The [Range] attribute enforces that a numeric (or date) property lies within a specified interval.
This is perfect for properties like ExperienceInYears, Salary, or Age, where values should be within logical bounds.

Example:
[Range(0, 40, ErrorMessage = "Experience must be between 0 and 40 years.")]
public int ExperienceInYears { get; set; }

You can also use [Range] for dates:
[Range(typeof(DateTime), "2020-01-01", "2030-12-31", ErrorMessage = "Joining Date must be between 2020 and 2030.")]
public DateTime JoiningDate { get; set; }

How it works: ASP.NET Core checks if the provided value lies within the defined boundaries. If it doesn't, it returns a validation error automatically.

[RegularExpression]

The [RegularExpression] attribute validates data using a **Pattern (Regex)**. It's the go-to solution for verifying specific input formats, such as **PIN Codes**, **PAN Numbers**, **Aadhaar Numbers**, or **Strong Passwords.**

Examples:

[RegularExpression(@"^\d{6}$", ErrorMessage = "Invalid ZIP Code. Must be 6 digits.")]
public string ZipCode { get; set; }

[RegularExpression(@"^[A-Z]{5}[0-9]{4}[A-Z]{1}$", ErrorMessage = "Invalid PAN format.")]
public string PanNumber { get; set; }

[RegularExpression(@"^\d{12}$", ErrorMessage = "Invalid Aadhaar number. Must be 12 digits.")]
public string AadhaarNumber { get; set; }

[RegularExpression(@"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{6,}$", ErrorMessage = "Password must contain at least one uppercase, one lowercase, one number, and one special character.")]
public string Password { get; set; }

Use Case Examples:

- Validate Indian PIN/ZIP codes (@"^\d{6}$")
- Validate Indian mobile numbers (@"^[6-9]\d{9}$")
- Validate strong passwords (as above)
- Validate PAN and Aadhaar numbers for official compliance

Why use it: Regex-based validation helps catch invalid data formats early, before they reach the database or trigger downstream errors.

[EmailAddress]

The [EmailAddress] attribute ensures that the given string follows a **Valid Email Address Format** (e.g., user@domain.com). It's widely used in login forms and user registration modules. It doesn't check whether the email actually exists, only the format.

```
Example:
[EmailAddress(ErrorMessage = "Invalid Email Address.")]
public string Email { get; set; }
```

How it works: ASP.NET Core internally uses the System.Net.Mail.MailAddress class to verify proper syntax.

[Compare]

The [Compare] attribute ensures that the values of two properties within the same model match. This is typically used for scenarios such as **confirming passwords**, **retyping email addresses,** or **entering credit card numbers**.

Example:

```
[Required(ErrorMessage = "Password is required.")]
public string Password { get; set; }

[Compare("Password", ErrorMessage = "Passwords do not match.")]
public string ConfirmPassword { get; set; }
```

How it works: ASP.NET Core compares both property values during model validation. If they differ, it returns a validation error, preventing users from accidentally mistyping passwords.

[Phone]

This attribute checks whether a string is in a **Valid Phone Number Format** according to the system's culture. It ensures basic structural correctness but doesn't verify if the number exists.

Example:

```
[Phone(ErrorMessage = "Invalid Phone Number.")]
public string PhoneNumber { get; set; }
```

For stricter Indian-specific validation, use:

```
[RegularExpression(@"^[6-9]\d{9}$", ErrorMessage = "Invalid Indian mobile number.")]
public string PhoneNumber { get; set; }
```

Why it matters: Phone number validation helps ensure proper communication channels and prevents bad data entry (like missing digits or letters).

[Url]

The [Url] attribute validates that a string contains a **Properly Formatted URL**, including a valid scheme like http or https.

Example:

```
[Url(ErrorMessage = "Invalid Website URL.")]
public string Website { get; set; }
```

Use Case: Useful for validating optional fields like **LinkedIn Profile**, **Portfolio Website**, or **Company Domain** during user registration.

[EnumDataType]

The [EnumDataType] attribute ensures that the value provided corresponds to a valid member of a defined enum type. It is especially useful for fields like Gender, Department, and AccountType in your model.

```
Example:
[EnumDataType(typeof(Gender), ErrorMessage = "Invalid Gender value.")]
public Gender Gender { get; set; }

[EnumDataType(typeof(Department), ErrorMessage = "Invalid Department value.")]
public Department Department { get; set; }

[EnumDataType(typeof(AccountType), ErrorMessage = "Invalid Account Type.")]
public AccountType AccountType { get; set; }
```

How it works: If a client tries to send an invalid enum string (e.g., “Gender”: “Unknown”), the model binder will reject the request with a 400 response.

Note: With JsonStringEnumConverter enabled in Program.cs, the API accepts and returns enum names like “Male”, “HR”, “Employee” instead of numeric values.

Implementing Server-Side Validations in ASP.NET Core Web API

When a client sends a request to the API (e.g., registering a user), we want to ensure the data they send is valid before saving or processing it. In ASP.NET Core, we can enforce these rules declaratively using **Data Annotation Attributes** on our model class. Let’s now create a simple working example step by step.

Create the Project

Create a new ASP.NET Core Web API project named **ValidationDemo**. Then, create a folder named **Models** in the project root directory.

Define the Model with Data Annotations

Create a class file named **User.cs** within the **Models** folder and then copy and paste the following code. The following class properties are decorated with various Data Annotation Attributes.

```
using System.ComponentModel.DataAnnotations;
namespace ValidationDemo.Models
{
```

```
    public class User
    {
        // Id is generated by the system (e.g., database auto-increment or GUID).
        // It is OPTIONAL while creating a new user because the client should not
supply it.
        // Hence, it's nullable and not decorated with [Required].
        public int? Id { get; set; }

        // MANDATORY: FirstName must always be provided when creating a user.
        // [Required] ensures the property is not null or empty.
        // [StringLength] restricts length between 3 and 50 characters for data
consistency.
        [Required(ErrorMessage = "First Name is required.")]
        [StringLength(50, MinimumLength = 3, ErrorMessage = "First Name must be
between 3 and 50 characters.")]
        public string FirstName { get; set; } = null!;

        // OPTIONAL: LastName is not required for single-name users.
        // [StringLength] ensures that if supplied, it does not exceed 50
characters.
        [StringLength(50, ErrorMessage = "Last Name cannot exceed 50
characters.")]
        public string? LastName { get; set; }

        // MANDATORY: Gender must be selected from predefined enum values (Male,
Female, Other).
        // [Required] ensures a value is provided, and [EnumDataType] validates
against the Gender enum.
        [Required(ErrorMessage = "Gender is required.")]
        [EnumDataType(typeof(Gender), ErrorMessage = "Invalid Gender value.")]
        public Gender Gender { get; set; }

        // MANDATORY: Email is critical for identification and communication.
        // [Required] makes it mandatory, and [EmailAddress] ensures a valid
email format (like user@example.com).
        [Required(ErrorMessage = "Email is required.")]
        [EmailAddress(ErrorMessage = "Invalid Email Address.")]
        public string Email { get; set; } = null!;

        // OPTIONAL: [Phone] validates format only if a value is provided.
        // Because it's not decorated with [Required], the client may skip it.
        // Example: A user can register without a phone number.
        [Phone(ErrorMessage = "Invalid Phone Number.")]
        public string? PhoneNumber { get; set; }

        // MANDATORY: DateOfBirth is required for eligibility and verification
purposes.
```

```
        // [Required] ensures it is always supplied.
        [Required(ErrorMessage = "Date of Birth is required.")]
        public DateTime DateOfBirth { get; set; }

        // MANDATORY: Department must be selected from a predefined list of valid
departments.
        // Using an Enum prevents invalid entries like "I.T" or "InfoTech".
        // [Required] ensures a value is provided, and [EnumDataType] validates
it against the Department enum.
        [Required(ErrorMessage = "Department is required.")]
        [EnumDataType(typeof(Department), ErrorMessage = "Invalid Department
value.")]
        public Department Department { get; set; }

        // OPTIONAL: Designation (like Developer, Manager, etc.) can be empty.
        // [StringLength] ensures it does not exceed 50 characters if provided.
        [StringLength(50, ErrorMessage = "Designation cannot exceed 50
characters.")]
        public string? Designation { get; set; }

        // MANDATORY: Represents total professional experience in years.
        // [Range] enforces acceptable limits (0–40) to avoid unrealistic inputs.
        [Range(0, 40, ErrorMessage = "Experience must be between 0 and 40
years.")]
        public int ExperienceInYears { get; set; }

        // MANDATORY: JoiningDate indicates when the employee/user joined or
registered.
        // [Required] ensures the value is always provided.
        [Required(ErrorMessage = "Joining Date is required.")]
        public DateTime JoiningDate { get; set; }

        // OPTIONAL: Address is not required, but [MaxLength] restricts the value
length if provided.
        // Useful for storing user address.
        [MaxLength(250, ErrorMessage = "Address cannot exceed 250 characters.")]
        public string? Address { get; set; }

        // OPTIONAL: City name. [StringLength] ensures proper formatting if
supplied.
        [StringLength(50, ErrorMessage = "City cannot exceed 50 characters.")]
        public string? City { get; set; }

        // OPTIONAL: Country name. [StringLength] ensures value remains within
expected length.
        [StringLength(50, ErrorMessage = "Country cannot exceed 50 characters.")]
        public string? Country { get; set; }
```

```
        // OPTIONAL: Postal/ZIP code of the user's location.
        // [RegularExpression] validates that if provided, it must contain
exactly 6 digits.
        [RegularExpression(@"^\d{6}$", ErrorMessage = "Invalid ZIP Code. Must be
6 digits.")]
        public string? ZipCode { get; set; }

        // OPTIONAL: PAN Number (for Indian context) used for identity
verification.
        // [RegularExpression] validates Indian PAN format (ABCDE1234F).
        [RegularExpression(@"^[A-Z]{5}[0-9]{4}[A-Z]{1}$", ErrorMessage = "Invalid
PAN format.")]
        public string? PanNumber { get; set; }

        // OPTIONAL: Aadhaar number (for Indian context), 12-digit format.
        // [RegularExpression] ensures only numeric input with exact 12 digits if
provided.
        [RegularExpression(@"^\d{12}$", ErrorMessage = "Invalid Aadhaar number.
Must be 12 digits.")]
        public string? AadhaarNumber { get; set; }

        // OPTIONAL: Personal or professional profile link (like LinkedIn or
website).
        // [Url] ensures valid URL format only when supplied.
        [Url(ErrorMessage = "Invalid URL.")]
        public string? Website { get; set; }

        // MANDATORY: Password is required for authentication.
        // [Required] ensures it's not empty.
        // [MinLength] enforces at least 6 characters.
        // [RegularExpression] ensures complexity: at least one uppercase, one
lowercase, one digit, and one special character.
        [Required(ErrorMessage = "Password is required.")]
        [MinLength(6, ErrorMessage = "Password must be at least 6 characters
long.")]
        [RegularExpression(@"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-
z\d@$!%*?&]{6,}$",
            ErrorMessage = "Password must contain at least one uppercase, one
lowercase, one number, and one special character.")]
        public string Password { get; set; } = null!;

        // MANDATORY: Must match the Password property value.
        // [Compare] ensures both password fields are identical.
        // Example: "Password" and "ConfirmPassword" must be the same.
        [Compare("Password", ErrorMessage = "Passwords do not match.")]
        public string? ConfirmPassword { get; set; }
```

```
        // MANDATORY: AccountType or role assigned to the user.
        // [EnumDataType] ensures value matches a valid AccountType (Employee,
Manager, Admin, HR).
        [EnumDataType(typeof(AccountType), ErrorMessage = "Invalid Account
Type.")]
        public AccountType AccountType { get; set; }
    }

    // Enum for Gender ensures users choose from defined values only.
    public enum Gender
    {
        Male,
        Female,
        Other
    }

    // Enum for AccountType ensures controlled input for roles or designations.
    public enum AccountType
    {
        Employee,
        Manager,
        Admin,
        HR
    }

    // Enum for Department ensures users select from valid predefined
departments.
    public enum Department
    {
        IT,
        HR,
        Finance,
        Sales,
        Marketing,
        Operations,
        Support
    }
}
```

Create the Controller

Next, we need to create a controller to handle HTTP requests and perform validations. So, create a new API Empty Controller named **UsersController** within the **Controllers** folder and then copy and paste the following code:

```
using Microsoft.AspNetCore.Mvc;
using ValidationDemo.Models;
```

```
namespace ValidationDemo.Controllers
{
    // [ApiController] automatically performs model validation and returns
    // 400 Bad Request responses if ModelState is invalid.
    [ApiController]
    [Route("api/[controller]")]
    public class UsersController : ControllerBase
    {
        // In-memory list acting as a mock database for demonstration.
        // When the application starts, it will already contain a few dummy
users.
        private static readonly List<User> Users = new()
        {
            new User {  Id = 1, FirstName = "Pranaya", LastName = "Rout", Gender
= Gender.Male,
                        Email = "pranayakumar777@gmail.com", PhoneNumber =
"9876543210",
                        DateOfBirth = new(1995,10,20), Department =
Department.IT, Designation = "Senior Software Engineer",
                        ExperienceInYears = 8, JoiningDate = new(2020,02,15),
Address = "Bhubaneswar, Odisha",
                        City = "Bhubaneswar", Country = "India", ZipCode =
"751024", PanNumber = "ABCDE1234F",
                        AadhaarNumber = "123412341234", Website =
"https://dotnettutorials.net/pranayarout",
                        Password = "Test@123", ConfirmPassword = "Test@123",
AccountType = AccountType.Employee },
            new User {  Id = 2, FirstName = "Ananya", LastName = "Mishra", Gender
= Gender.Female,
                        Email = "ananya.mishra@example.com", PhoneNumber =
"9876500000",
                        DateOfBirth = new(1997,04,05), Department =
Department.HR, Designation = "HR Executive",
                        ExperienceInYears = 3, JoiningDate = new(2022,06,10),
Address = "Cuttack, Odisha",
                        City = "Cuttack", Country = "India", ZipCode = "753001",
PanNumber = "PQRSX6789K",
                        AadhaarNumber = "567856785678", Website =
"https://dotnettutorials.net/ananyamishra",
                        Password = "Pass@2024", ConfirmPassword = "Pass@2024",
AccountType = AccountType.HR }
        };

        // POST: api/Users
        // Creates a new user and validates the input using data annotations.
        [HttpPost]
```

```
        public IActionResult CreateUser([FromBody] User user)
        {
            // [ApiController] already performs model validation automatically,
            // but we include this explicit check for clarity and control.
            if (!ModelState.IsValid)
            {
                // Returns HTTP 400 with detailed validation errors.
                return BadRequest(ModelState);
            }

            // Assign a new unique Id (simulating auto-increment from a
database).
            user.Id = Users.Max(u => u.Id ?? 0) + 1;

            // Add the new user to the in-memory list.
            Users.Add(user);

            // Returns HTTP 201 (Created) with Location header.
            // The response body contains the newly created user details.
            return CreatedAtAction(nameof(GetUserById), new { id = user.Id },
user);
        }

        // GET: api/Users/{id}
        // Fetches a user based on the given Id.
        [HttpGet("{id}")]
        public IActionResult GetUserById(int id)
        {
            // Searches the in-memory list for the requested user.
            var user = Users.FirstOrDefault(u => u.Id == id);

            if (user == null)
            {
                // Returns 404 Not Found if the user does not exist.
                return NotFound(new { Message = $"User with Id {id} not found."
});
            }

            // Returns 200 OK with the matching user details.
            return Ok(user);
        }

        // GET: api/Users
        // Returns the list of all users.
        [HttpGet]
        public IActionResult GetAllUsers()
        {
```

```
            // If the list is empty, you can choose to return NoContent()
instead.
            if (!Users.Any())
            {
                // Returns 204 No Content when there are no users to display.
                return NoContent();
            }

            // Returns 200 OK with the list of users.
            return Ok(Users);
        }
    }
}
```

Configure the Application

Ensure that the application is set up to handle JSON requests and responses properly, and that validation errors are returned in a readable format. So, please modify the Program class as follows:

```
using System.Text.Json.Serialization;
namespace ValidationDemo
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Add services to the container.
            builder.Services.AddControllers()
            .AddJsonOptions(options =>
            {
                // This will use the property names as defined in the C# model
                options.JsonSerializerOptions.PropertyNamingPolicy = null;

                // Enable enum values to be read/written as strings instead of
numbers
                options.JsonSerializerOptions.Converters.Add(new
JsonStringEnumConverter());
            });

            builder.Services.AddEndpointsApiExplorer();
            builder.Services.AddSwaggerGen();

            var app = builder.Build();

            // Configure the HTTP request pipeline.
```

```
            if (app.Environment.IsDevelopment())
            {
                app.UseSwagger();
                app.UseSwaggerUI();
            }

            app.UseHttpsRedirection();

            app.UseAuthorization();

            app.MapControllers();

            app.Run();
        }
    }
}
```

How Validation Works Internally

1. When the client sends a POST request to **/api/Users** with a JSON body, ASP.NET Core automatically binds the JSON data to the User model.
2. It checks all validation attributes during model binding.
3. If **Any Attribute Fails**, it adds an error to the ModelState.
4. Because of the [ApiController] attribute, ASP.NET Core automatically:
   - Returns a **400 Bad Request**
   - Includes detailed validation error messages in the response body.

Testing the Validations

You can test the API using tools like Postman or directly via Swagger UI.

Create User – Sample Request (POST /api/Users)

Here's a valid JSON request body for creating a new user:

```
{
  "FirstName": "Rahul",
  "LastName": "Patnaik",
  "Gender": "Male",
  "Email": "rahul.patnaik@example.com",
  "PhoneNumber": "9876543219",
  "DateOfBirth": "1998-08-25",
  "Department": "Finance",
  "Designation": "Accountant",
  "ExperienceInYears": 4,
```

```
  "JoiningDate": "2024-04-15",
  "Address": "Bhubaneswar, Odisha",
  "City": "Bhubaneswar",
  "Country": "India",
  "ZipCode": "751024",
  "PanNumber": "ASDFG1234H",
  "AadhaarNumber": "987698769876",
  "Website": "https://dotnettutorials.net/rahulpatnaik",
  "Password": "Finance@123",
  "ConfirmPassword": "Finance@123",
  "AccountType": "Employee"
}
```

Invalid Create User Request Example (POST /api/Users)

Here's a sample invalid request where multiple validation rules fail:

```
{
  "FirstName": "",
  "Email": "wrongemailformat",
  "Gender": "Male",
  "Password": "abc",
  "ConfirmPassword": "xyz"
}
```

Response – 400 Bad Request (Validation Errors Automatically Returned by ASP.NET Core).

400 *Undocumented*

Error: response status is 400

Response body


```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Email": [
      "Invalid Email Address."
    ],
    "Password": [
      "Password must be at least 6 characters long.",
      "Password must contain at least one uppercase, one lowercase, one number, and one special character."
    ],
    "FirstName": [
      "First Name is required.",
      "First Name must be between 2 and 50 characters."
    ],
    "ConfirmPassword": [
      "Passwords do not match."
    ]
  },
  "traceId": "00-56ce4c95daa9cc8741b6cb6aac39cb9c-6f42e84c58c424e2-00"
}
```


Note: The traceId field is auto-generated by ASP.NET Core and will differ for every request.

Custom Data Annotation Attribute in ASP.NET Core Web API

In ASP.NET Core Web API, **Data Annotation Attributes** are commonly used to validate model properties before the data reaches your business logic or database. The framework provides several **Built-In Attributes**, such as [Required], [StringLength], [Range], [EmailAddress], etc., that cover most standard validation scenarios.

However, in Real-World Applications, we often encounter **Business-Specific Validation Requirements** that go beyond the capabilities of these built-in attributes. For example, suppose we have a **Date of Birth (DOB)** field in our model and want to ensure that the user's **Age Is at Least 18 Years**.

Unfortunately, there is **No Built-In Data Annotation Attribute** in ASP.NET Core that can validate age based on the date of birth directly. This is where **Custom Data Annotation Attributes** come into play. They allow us to **define our own Validation Logic** that precisely fits our business or domain requirements.

Why We Need Custom Data Annotations

Built-in attributes are great for common use cases such as "Field Required," "String Too Long," and "Number Within Range." But, in many real-world scenarios, validations depend on **Custom Logic**, such as:

- Checking that the employee's age is above 18.
- Ensuring a joining date is after the date of birth.
- Ensuring the end date is greater than the **start date** in booking systems.
- Restricting uploads to specific file extensions or sizes.
- Verifying a username is unique in the database.
- Enforce that a leave application does not exceed the employee's remaining leave balance.

To implement these validations in a clean, reusable way, we create **Custom Validation Attributes** in ASP.NET Core.

Steps to Create a Custom Data Annotation Attribute

Implementing a Custom Validation Attribute in ASP.NET Core is a **Three-Step Process**. They are as follows.

Step 1: Inherit from ValidationAttribute

Create a new class that inherits from **System.ComponentModel.DataAnnotations.ValidationAttribute** base class. This gives your custom class access to the built-in validation infrastructure.

Step 2: Override the IsValid() Method

The Core Custom Validation Logic is implemented by **overriding the IsValid() Method**. This method receives the property value as a parameter and returns either:

- **ValidationResult.Success** if validation passes, or
- A new **ValidationResult** instance containing an error message if validation fails.

Inside this method, you can:

- Perform calculations (like age computation from DateOfBirth).
- Cross-check with the current date/time.
- Apply any domain-specific rule.

Step 3: Apply the Custom Attribute to the Model Property

Once the attribute is created, you simply decorate the relevant model property (like DateOfBirth) with it, just like any built-in validation attribute.

Example to Understand Custom Validation Attribute

When registering a user or adding an employee, we may want to ensure that the **calculated age from their Date of Birth** falls between **18 and 60 years**.

ASP.NET Core doesn't include a built-in attribute for age validation; that's where **Custom Validation Attributes** come into play. They allow you to encapsulate complex validation logic in a reusable, declarative, and maintainable way.

Step 1: Creating the Custom Validation Attribute

To maintain clean project organization, create a new folder named **Validators** in your project root. Then, create a class file named **AgeValidationAttribute.cs** within the **Validators** folder and copy and paste the following code:

```
using System.ComponentModel.DataAnnotations;
namespace ValidationDemo.Validators
{
    // Custom validation attribute to ensure that the user's age (calculated from
Date of Birth)
    // falls within a specific range.
    // Example usage:
    //     [AgeValidation(18, 60, ErrorMessage = "Age must be between 18 and 60
years.")]
    //     public DateTime DateOfBirth { get; set; }
    public class AgeValidationAttribute : ValidationAttribute
    {
```

```
        private readonly int _minimumAge;
        private readonly int _maximumAge;

        // minimumAge: The minimum allowable age (default: 18).
        // maximumAge: The maximum allowable age (default: 60).
        public AgeValidationAttribute(int minimumAge = 18, int maximumAge = 60)
        {
            _minimumAge = minimumAge;
            _maximumAge = maximumAge;

            // Default error message if not overridden during attribute usage
            ErrorMessage = $"Age must be between {_minimumAge} and {_maximumAge}
years.";
        }

        // Performs the actual validation logic.
        // Parameters:
        //      value: The property value being validated (expected: DateTime).
        //      validationContext: Contextual information about the validation
process.
        // returns:
        //      Returns a ValidationResult with an error message if validation
fails,
        //      or ValidationResult.Success if validation passes.
        protected override ValidationResult? IsValid(object? value,
ValidationContext validationContext)
        {
            // If no value is provided, validation fails immediately.
            // The [Required] attribute should ideally handle this, but we check
it as a fallback.
            if (value == null)
            {
                return new ValidationResult("Date of Birth is required.");
            }

            // Ensure that the incoming value is a valid DateTime type.
            // If not, return an explicit validation failure message.
            if (value is not DateTime dateOfBirth)
            {
                return new ValidationResult("Invalid Date of Birth format.");
            }

            // Get today's date for age calculation.
            var today = DateTime.Today;

            // Basic age calculation: difference in years between today and birth
year.
```

```
            int age = today.Year - dateOfBirth.Year;

            // Adjust the calculated age if the birthday hasn't occurred yet in
the current year.
            // Example: If today is March and the user's birthday is in July,
reduce age by 1.
            if (dateOfBirth.Date > today.AddYears(-age))
                age--;

            // Validate the computed age against the specified range.
            if (age < _minimumAge || age > _maximumAge)
            {
                // Return a descriptive validation error if the user's age is out
of range.
                return new ValidationResult(ErrorMessage);
            }

            // If all checks pass, return success (indicating validation passed).
            return ValidationResult.Success;
        }
    }
}
```

Understanding How This Works

This class **inherits from ValidationAttribute**, which is part of the ASP.NET Core **DataAnnotations** namespace. By overriding the IsValid() method, we insert our own validation logic into ASP.NET Core's **model validation pipeline**. When the framework validates an incoming request:

1. It examines the model's properties.
2. It detects this custom [AgeValidation] attribute.
3. It executes the IsValid() method.
4. Based on the result, it either:
    - Marks the property as valid
    - Or returns a ValidationResult with your error message.

This happens **before the controller action executes**, ensuring invalid data never reaches your business logic.

What is ValidationContext?

In ASP.NET Core, **ValidationContext** is a class that provides **Contextual Information** about the object being validated when a validation attribute is executed. When the framework validates our model, it passes a ValidationContext instance to your IsValid() method. This object gives you access to:

- The **Entire Object** (model) is being validated, not just the property value.
- The **Type** and **Name** of the member being validated.
- **External Services** (via dependency injection) you can use for advanced validation (e.g., database checks).

For example: var user = validationContext.ObjectInstance as User; This lets you access the complete User model inside your custom validator.

Step 2: Applying the Custom Attribute to the User Model

Once your custom validator is ready, apply it to the DateOfBirth property inside your User model as follows:

[Required(ErrorMessage = “Date of Birth is required.”)]
[AgeValidation(18, 60)]
public DateTime DateOfBirth { get; set; }

Here’s what happens:

- [Required] ensures that DateOfBirth is not null.
- [AgeValidation(18, 60)] ensures that the calculated age falls within the range of **18 to 60 years**.

Example Request That Passes

```
{
  "FirstName": "Ananya",
  "Gender": "Female",
  "Email": "ananya@example.com",
  "DateOfBirth": "1995-05-10",
  "Department": "IT",
  "JoiningDate": "2024-06-01",
  "Password": "Secure@123",
  "ConfirmPassword": "Secure@123",
  "AccountType": "Employee"
}
```

Result: Validation passes successfully. The user is 30 years old, which is within the allowed range (18–60).

Example Request That Fails:

```
{
  "FirstName": "Raj",
  "Gender": "Male",
  "Email": "raj@example.com",
  "DateOfBirth": "2010-08-15",
  "Department": "HR",
```

```
  "JoiningDate": "2025-01-01",
  "Password": "Pass@123",
  "ConfirmPassword": "Pass@123",
  "AccountType": "Employee"
}
```

Error Response:

400
Undocumented

Error: response status is 400

Response body


```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "DateOfBirth": [
      "Age must be between 18 and 60 years."
    ]
  },
  "traceId": "00-9506bb0c4988956fdbdd7b3de3516687-33b29a3fc766db19-00"
}
```


Benefits of Using Custom Attributes

1. **Reusable**: Once created, you can apply **[AgeValidation(18, 60)]** anywhere, for users, drivers, customers, etc. The value might vary from place to place.
2. **Centralized Logic**: Validation logic is encapsulated and reusable, keeping controllers and services clean.
3. **Consistency**: Prevents duplicating age-check logic across different layers.
4. **Integration-Friendly**: Works seamlessly with [ApiController], provides automatic validation, and integrates with Swagger UI.

How to Return a Custom 400 Response when Model Validation Fails?

By default, ASP.NET Core (with [ApiController]) automatically returns a **400 Bad Request** when model validation fails, but the response shape might not match what you want (for example, you might want to include your own message, status code format, or field-level details).

Default Behavior (Without Customization)

When we decorate our controller with [ApiController], ASP.NET Core **automatically** performs model validation before executing your action method. If validation fails, it returns a **400 Bad Request** response by creating an instance of the ProblemDetails class as follows:

```
{
```

```
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Email": [
      "Invalid Email Address."
    ],
    "Password": [
      "Password must be at least 6 characters long.",
      "Password must contain at least one uppercase, one lowercase, one number,
and one special character."
    ],
    "ConfirmPassword": [
      "Passwords do not match."
    ]
  },
  "traceId": "00-b71f4a1f38818b3684e6c683f487ceb4-3a38ab86d9abb2d9-00"
}
```

This is fine for debugging, but **Not Ideal** for real-world APIs, especially if you want consistent, human-friendly, or frontend-ready error responses. We will now customize this behaviour to return our **Own 400 Response** format, something like this:

```
{
  "Success": false,
  "StatusCode": 400,
  "Message": "Validation failed. Please correct the highlighted errors.",
  "Errors": {
    "Email": [
      "Invalid Email Address."
    ],
    "Password": [
      "Password must be at least 6 characters long.",
      "Password must contain at least one uppercase, one lowercase, one number,
and one special character."
    ],
    "ConfirmPassword": [
      "Passwords do not match."
    ]
  }
}
```

This is more descriptive, compact, and user-friendly. Let us proceed and see how we can implement this in ASP.NET Core Web API.

Custom Model Validation Response in ASP.NET Core Web API

To return a custom 400 Error Response instead of the default Problem Details response, we need to configure the **InvalidModelStateResponseFactory** inside the **ConfigureApiBehaviorOptions** method. This factory allows us to **override the default automatic validation response**. This approach:

- Runs **only** for **Automatic [ApiController]** validation failures.
- Does **not** interfere with manually returned **BadRequest()** responses.

Please modify the Program class as follows.

```
using Microsoft.AspNetCore.Mvc;
using System.Text.Json.Serialization;

namespace ValidationDemo
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // ------------------------------------------------------------
            // Configure Services
            // ------------------------------------------------------------
            // The AddControllers() method registers the Controllers and
            // enables API-related features like model binding and validation.
            // Here, we also customize the JSON serialization and model
validation behavior.
            builder.Services.AddControllers()

            // ------------------------------------------------------------
            // Customize JSON Serialization
            // ------------------------------------------------------------
            .AddJsonOptions(options =>
            {
                // By default, System.Text.Json converts property names to
camelCase.
                // Setting this to null ensures property names remain as defined
in your C# models.
                options.JsonSerializerOptions.PropertyNamingPolicy = null;

                // This ensures that enums are serialized and deserialized as
their string names
                // instead of numeric values.
                options.JsonSerializerOptions.Converters.Add(new
JsonStringEnumConverter());
            })
```

```
            // ------------------------------------------------------------
            // Customize Automatic Model Validation Response
            // ------------------------------------------------------------
            // The [ApiController] attribute automatically validates incoming
request models.
            // By default, ASP.NET Core returns a 400 Bad Request response
containing a ProblemDetails object
            // when model validation fails (e.g., missing required fields,
invalid formats).
            //
            // The following customization overrides that default behavior to
produce a
            // consistent, developer-friendly JSON response format that aligns
with a unified API contract.
            //
            .ConfigureApiBehaviorOptions(options =>
            {
                //The InvalidModelStateResponseFactory is invoked only when
ModelState.IsValid is false.
                //This means it's specifically designed to handle and format
responses for requests that failed model validation.
                //It will include those properties for which Model validation is
Failed.
                options.InvalidModelStateResponseFactory = context =>
                {
                    // ------------------------------------------------------------
-----
                    // Step 1: Initialize a collection to hold validation errors
                    // ------------------------------------------------------------
-----
                    // Using a dictionary ensures that each field name (key)
appears only once,
                    // even if multiple validation rules fail for the same
property.
                    var errors = new Dictionary<string, List<string>>();

                    // ------------------------------------------------------------
-----
                    // Step 2: Iterate through each entry in the ModelState
                    // ------------------------------------------------------------
-----
                    // The ModelState contains one entry per bound property or
field in the model.
                    // Each entry may contain one or more validation errors.
                    foreach (var kvp in context.ModelState)
                    {
```

```
                        var fieldName = kvp.Key;   // The name of the field or
property being validated
                        var entry = kvp.Value;     // The validation state and
error collection for that field

                        // Skip processing if the field has no validation errors
                        if (entry == null || entry.Errors.Count == 0)
                            continue;

                        // ------------------------------------------------------
---------
                        // Step 3: Ensure a list exists for the current field
                        // ------------------------------------------------------
---------
                        // This allows grouping multiple error messages under a
single field name.
                        if (!errors.ContainsKey(fieldName))
                            errors[fieldName] = new List<string>();

                        // ------------------------------------------------------
---------
                        // Step 4: Extract and normalize all error messages for
this field
                        // ------------------------------------------------------
---------
                        // Each ModelError may contain a specific validation
message.
                        // If an error message is missing or blank, we fall back
to a generic "Invalid value."
                        foreach (var error in entry.Errors)
                        {
                            var message =
string.IsNullOrWhiteSpace(error.ErrorMessage)
                                ? "Invalid value."  // Default fallback message
                                : error.ErrorMessage;

                            errors[fieldName].Add(message);
                        }
                    }

                    // ------------------------------------------------------------
-----
                    // Step 5: Construct a standardized API response object
                    // ------------------------------------------------------------
-----
                    // The goal is to keep a consistent structure across all
validation failures,
```

```
                    // making it easier for client applications (web/mobile) to
handle and display errors.
                    var response = new
                    {
                        Success = false, // Indicates that the operation failed
                        StatusCode = 400, // HTTP 400 Bad Request
                        Message = "Validation failed. Please correct the
highlighted errors.",
                        Errors = errors   // Dictionary of field names and their
respective error messages
                    };

                    // ------------------------------------------------------------
-----
                    // Step 6: Return the structured 400 BadRequest response
                    // ------------------------------------------------------------
-----
                    // Using BadRequestObjectResult automatically sets the
response status to 400.
                    // Explicitly defining the content type ensures JSON
serialization consistency.
                    return new BadRequestObjectResult(response)
                    {
                        ContentTypes = { "application/json" } // Ensure the
response body is JSON formatted
                    };
                };
            });

            builder.Services.AddEndpointsApiExplorer();
            builder.Services.AddSwaggerGen();
            var app = builder.Build();

            if (app.Environment.IsDevelopment())
            {
                app.UseSwagger();
                app.UseSwaggerUI();
            }

            app.UseHttpsRedirection();
            app.UseAuthorization();
            app.MapControllers();

            app.Run();
        }
    }
}
```

Note: The InvalidModelStateResponseFactory is invoked only when ModelState.IsValid is false. This means it's specifically designed to handle and format responses for requests that failed model validation. It will include those properties for which the Model validation has failed.

Example Invalid Request

```
{
  "FirstName": "",
  "Email": "invalid-email",
  "DateOfBirth": "2010-05-10",
  "Password": "weak",
  "ConfirmPassword": "wrong"
}
```

Data Annotation Attributes are one of the **Simplest and Most Powerful Ways** to apply server-side validation in ASP.NET Core Web API. They:

- Reduce duplicate code
- Keep validation rules close to the data
- Provide consistent error messages
- Work seamlessly with automatic model validation ([ApiController])

However, for **Complex** or **Conditional** validations (e.g., dependent fields or dynamic business rules), you should combine them with **FluentValidation** or **Custom Logic** in your service layer.

In the next article, I will discuss **Automapper in ASP.NET Core Web API** with Examples. In this article, I explain how to implement Server-side Validation using data annotations in ASP.NET Core Web API with examples. I hope you enjoy this article, "Validation using data annotations in ASP.NET Core Web API."

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.